MANTHAN 2026: 24-HOUR HACKATHON

Jawaharlal Nehru New College of Engineering (JNNCE), Shivamogga
Department of Computer Science & Engineering (Data Science)
Autonomous Trading Systems Challenge

[STEP 1: MANDATORY VENUE WI-FI CONNECTION] All participant laptops must connect to the official venue Wi-Fi network. The entire competition ecosystem—including the Kafka market stream, Order Gateway APIs, and Live Telemetry Dashboards—runs exclusively inside this local private network. You cannot connect to the competition servers using mobile hotspots or external internet connections. Obtain the Wi-Fi SSID and password from the registration desk.

[OFFICIAL VIDEO BRIEFING & STRATEGY WALKTHROUGH] Watch the official 9-minute briefing video explaining competition rules, market simulation mechanics, pre-flight heartbeat requirements, and platform workflow:

Official Briefing Video URL: https://youtu.be/QKgdReXq1-M *(Embedded directly in the Participant Web Portal under the 'Participant Guide' page).*

Key Highlights Covered in the Video:
- Real-time market tick and corporate news consumption via Apache Kafka.
- Dynamic stock universe discovery via GET /api/v1/stocks.
- Mandatory pre-flight sequence heartbeat acknowledgment via POST /api/v1/bot/heartbeat.
- 4 supported order actions: BUY, SELL, SHORT, COVER.
- Critical decision latency constraint: Your bot must make trading decisions within 3 to 4 seconds.
- Virtual starting capital of ₹10,00,000.00 and monitoring portfolio metrics via the live dashboard.

Hackathon Parameter	Details
Event Name	MANTHAN 2026 (24-Hour National Level Hackathon)
Organizing Department	Department of CSE (Data Science), JNNCE, Shivamogga
Domain	Autonomous Trading Systems ('Churn Data, Discover Truth')
Total Prize Pool	₹1,00,000 (Awards for 1st, 2nd, 3rd, and top category teams)
Team Size	2 to 4 members per team
Starting Virtual Capital	₹10,00,000.00 (Ten Lakhs INR) credited to each team account
Market Data Stream	Apache Kafka on <SERVER_IP>:9094 (External Broker Port, PLAINTEXT)
Order Execution API	REST API on http://<SERVER_IP>:3002/api/v1
Participant Portal & Docs	http://<SERVER_IP>:3003 (Live Dashboard, Participant Guide & API Directory)
Official Briefing Video	https://youtu.be/QKgdReXq1-M (Embedded in Participant Guide UI)

1. Problem Statement: What You Have to Build

Your challenge is to design, build, and deploy an autonomous trading bot.

During the hackathon, our platform replays historical stock market data in real time. Your bot will continuously ingest price candles and corporate news announcements, make automated trading decisions using your custom strategies (heuristics, quantitative rules, technical indicators, machine learning, or sentiment analysis), and submit orders to our exchange server using REST APIs.

What Your Bot Must Do

- **1. Ingest Streaming Market Data:** Subscribe to the Kafka broker on topics 'market.ticks' and 'market.news'. Ingest daily OHLCV candlestick records and corporate news flashes.
- **2. Make Real-Time Trading Decisions:** Analyze incoming market data within a strict 3- to 4-second decision window. Apply quantitative indicators, machine learning models, or news sentiment to decide whether to BUY, SELL, SHORT, COVER, or DO NOTHING.
- **3. Execute Orders Cleanly:** Submit market orders through our REST API at http://<SERVER_IP>:3002/api/v1/orders with your Bearer token. Handle confirmations and rejections cleanly.
- **4. Manage Portfolio & Risk:** Track cash balances, monitor stock holdings, manage margin for short positions, minimize transaction costs, and prevent negative-balance account blocks.

Technology & Strategic Freedom

- **Any Programming Language:** You can build your bot in any programming language (Python, JavaScript/Node.js, TypeScript, Go, Java, C++, Rust, etc.). Ready-to-run starter templates are available directly on the Participant Portal.
- **Any Libraries & Frameworks:** You are encouraged to use open-source libraries (pandas, numpy, ta-lib, scikit-learn, PyTorch), technical indicators (Moving Averages, RSI, MACD, Bollinger Bands), statistical arbitrage, or mean-reversion logic.
- **AI & Generative AI Tools Welcome:** You are fully permitted to use ChatGPT, GitHub Copilot, Cursor, and pre-trained NLP/machine learning models to accelerate coding, analyze corporate news sentiment, and optimize trading strategies.
- **Discipline Over Hyperactivity:** A disciplined bot recognises when market conditions are unfavorable. 'Doing nothing' is a valid and often optimal strategic decision that preserves capital.

> **[CONFIDENTIAL BLIND MARKET REPLAY]** The historical time period being replayed is completely confidential. Stock symbols are also masked with virtual identifiers (e.g., ALPHACORP, BETATECH). You cannot look up future prices online. Your bot must react dynamically to the live stream.

2. Hackathon Schedule & How Evaluation Works

The hackathon follows a structured 24-hour timeline. Setup and verification take strictly 30 minutes at the start.

Stage	Timeline	What Happens
Setup & Verification	0:00 – 0:30 (First 30 Mins)	Connect to venue Wi-Fi, sign up on the portal, verify at the registration desk, and watch the official briefing video.

Stage	Timeline	What Happens
Development & Sandbox Testing	0:30 – 20:00 (Hours 0.5 to 20)	Build your bot, connect to Kafka, backtest against the continuous practice stream, and test API orders.
Checkpoint Reviews	During Development	Jury rounds to review your system architecture, algorithmic logic, risk controls, and early progress.
Pre-Flight & 1-Hour Live Simulation	Hours 20:00 – 23:00	Pre-flight readiness checks followed by the official 1-hour live market simulation. Bots trade autonomously.
Final Round Jury Defense (Top 5 Selection)	Hours 23:00 – 24:00	Top qualifying teams defend their code, system architecture, and financial strategy before the jury across core evaluation pillars (Section 9) to select the final winners.

[EVALUATION & SHORTLISTING POLICY] HOW WINNERS ARE CHOSEN (CRITICAL NOTICE):

Winners are NOT selected solely by who made the highest gross profit!

Following the 1-hour live simulation, the platform calculates a comprehensive scorecard and shortlists qualifying teams based on balanced metrics: Total Wealth, Cash Utilization, Decision Latency, Low Order Error Rates, and Tax & Friction Efficiency.

The jury panel then conducts an in-depth code review and defense interview with each finalist team based on the evaluation pillars detailed in Section 9. Judges will review your system architecture, mathematical logic, risk mitigation, and exception handling. A team with robust engineering and sound financial logic will win over a team that took reckless, unhedged gambles.

3. Market Data Stream Setup (Apache Kafka)

Market events are broadcast using Apache Kafka running on the venue local subnet.

Connection Coordinates

- **Kafka Broker:** <SERVER_IP>:9094 (External listener port on the local network; broker IP announced at the venue).
- **Security Protocol:** PLAINTEXT protocol. No username or password required on the competition subnet.
- **Consumer Group:** group-<your_team_username> (e.g., group-alpha_lead). Using this exact group prefix ensures the platform's automated Pre-Flight Readiness Tracker detects your consumer process.
- **Offset Strategy (Zero Overhead):** Configure auto_offset_reset='latest' and enable_auto_commit=False (already pre-configured in all official Starter Templates). You do NOT need to build an offset database or store offsets. Disabling auto-commit ensures Kafka never persists stale offsets, so every time your bot starts or reconnects, Kafka automatically streams strictly live incoming ticks and skips any offline backlog.

[PERFORMANCE ARCHITECTURE: UP TO 10 PARALLEL CONSUMERS] PRO-TIP: SCALING UP TO 10 PARALLEL

KAFKA CONSUMERS (10 TOPIC PARTITIONS):

Both the 'market.ticks' and 'market.news' topics are configured with 10 partitions on the Kafka broker.

In Apache Kafka, topic partitions are dynamically balanced across consumers sharing the exact same group ID (group-<your_team_username>):

• Single-Consumer Bottleneck: A single consumer process must sequentially read and process ticks across all 10 partitions. If your feature engineering, technical indicator updates, or order requests take computation time, a single thread risks lagging behind the 3- to 4-second real-time window.

• Up to 10x Parallel Throughput: Teams can run up to 10 parallel consumer instances (or worker processes/threads) within their assigned group ID (group-<your_team_username>). Kafka automatically balances 1 partition to each consumer. This distributes multi-stock tick ingestion in parallel with zero lag!

NOTE: Running more than 10 consumers will cause the 11th+ consumer to remain idle, since a partition cannot be assigned to more than one consumer in the same group.

Simulation Timing & Cadence

- **Daily Speed:** 1 simulated trading day equals about 3 to 4 seconds of real clock time.
- **T + 0 Seconds:** Daily OHLCV candlestick records for all stocks arrive on topic 'market.ticks'.
- **T + 150 Milliseconds:** Corporate news headlines and press announcements arrive on topic 'market.news'.
- **Decision Latency Window:** Your bot has the remaining ~3 seconds to process data and send orders before the next day begins. A brilliant decision that arrives too late is a missed opportunity.

Kafka Topic	Timing	Event Type	Payload Description
market.ticks	T + 0s (Daily Start)	price_tick	Daily OHLCV candle for all stocks: ticker, open, high, low, close, volume, simulated_date.
market.ticks	Pre-Flight Phase	preflight_tick	Standby readiness ping containing sequence number 'seq'. Requires bot heartbeat ACK.
market.news	T + 150ms	news_flash	Corporate press releases and disclosures: ticker, headline, description, date.

Sample price_tick Message (market.ticks)

```
{
  "event_type": "price_tick",
  "timestamp": "2026-10-19T10:00:00.000Z",
  "simulated_date": "2021-04-15",
  "data": {
    "ticker": "ALPHACORP",
```

```
    "open": 1440.00,
    "high": 1465.50,
    "low": 1435.20,
    "close": 1450.25,
    "volume": 1285000
  }
}
```

Sample news_flash Message (market.news)

```
{
  "event_type": "news_flash",
  "timestamp": "2026-10-19T10:00:00.150Z",
  "simulated_date": "2021-04-15",
  "data": {
    "ticker": "ALPHACORP",
    "headline": "ALPHACORP Q3 NET PROFIT SURGES 24% YOY",
    "description": "Company announces record exports and declares interim dividend...",
    "date": "2021-04-15"
  }
}
```

4. Pre-Flight Readiness Phase & Dynamic Stock Discovery

Before the official live simulation begins, the platform enters the Pre-Flight Stage. This stage guarantees that every team's bot process is running, connected to Kafka, and actively responding.

The Pre-Flight Heartbeat Protocol (Mandatory)

- **1. Listen for Standby Ticks:** During pre-flight, the server broadcasts 'preflight_tick' events on topic 'market.ticks'. Each tick contains a sequence number ('seq').
- **2. Continuous Sequence ACK:** As long as pre-flight ticks arrive, your bot MUST continuously acknowledge each sequence number by sending a POST request to /api/v1/bot/heartbeat.
- **3. 15-Second Window:** The platform requires a sequence acknowledgment at least once every 15 seconds. If received and Kafka is connected, your team status changes to 'Ready' on the portal dashboard.

Sample preflight_tick Event (market.ticks)

```
{
  "event": "preflight_tick",
  "event_type": "preflight_tick",
  "timestamp": "2026-10-20T10:00:00.000Z",
  "seq": 14,
  "data": {
    "status": "STANDBY",
    "seq": 14
  }
}
```

Pre-Flight Heartbeat Acknowledgment (POST /api/v1/bot/heartbeat)

```
POST http://<SERVER_IP>:3002/api/v1/bot/heartbeat
Authorization: Bearer <your_access_token>
Content-Type: application/json

{
  "seq": 14
}
```

Dynamic Stock Universe Discovery (GET /api/v1/stocks)

Do NOT hardcode stock ticker symbols in your bot. During the pre-flight phase, call GET /api/v1/stocks with your Bearer token. This endpoint returns the list of active tradable ticker symbols, company names, and business sector descriptions for the upcoming simulation run.

```
GET http://<SERVER_IP>:3002/api/v1/stocks
Authorization: Bearer <your_access_token>

// Response:
{
  "status": "ok",
  "stocks": [
    {
      "ticker": "ALPHACORP",
      "symbol": "ALPHACORP",
      "name": "Alpha Corporation Limited",
      "description": "Leading diversified manufacturing and industrial conglomerate..."
    },
    {
      "ticker": "BETATECH",
      "symbol": "BETATECH",
      "name": "Beta Technologies Global Inc.",
      "description": "Enterprise cloud infrastructure, cybersecurity, and data analytics
provider..."
    }
  ]
}
```

Simulation Lifecycle Stage Query (GET /api/v1/stage)

Bots need to know exactly when the platform transitions from Pre-Flight into the official 1-Hour Live Simulation. Instead of guessing or sending orders blindly, call GET /api/v1/stage (Public, no auth required) at zero delay to monitor the simulation state:

```
GET http://<SERVER_IP>:3002/api/v1/stage

// Response:
{
  "stage": "simulation",      // "development" | "ready" | "preflight" | "simulation"
  "status": "RUNNING",         // "IDLE" | "RUNNING" | "PAUSED" | "COMPLETED"
```

```
  "has_run": true,
  "simulation_id": "sim_nse_2026",
  "interval_seconds": 4,
  "total_days": 30,
  "days_completed": 12,
  "current_date": "2021-01-16"
}
```


- **Automated Launch:** Poll this endpoint every 2 to 3 seconds during pre-flight. The exact moment stage === 'simulation' and status === 'RUNNING', activate your automated trading order logic.
- **Graceful Execution:** If status changes to 'PAUSED' or 'COMPLETED', immediately pause order submissions to avoid SIMULATION_NOT_RUNNING error rejections.
- **Simulation Progress:** Track days_completed and total_days to manage end-of-simulation portfolio unwinding if your strategy requires closing positions.

5. Trading & Order REST APIs

All trading orders and account queries are executed via HTTP REST on Port 3002.

- **Base URL:** http://<SERVER_IP>:3002/api/v1
- **Authorization Header:** All authenticated endpoints require: Authorization: Bearer <your_jwt_token>

[PERSISTENT TOKEN CONFIGURATION] TOKEN MANAGEMENT BEST PRACTICE:

Your persistent access token is provided directly on the Participant Web Portal. Copy your access token from Step 1 of the Participant Guide and pass it to your bot as an environment variable (BOT_ACCESS_TOKEN) or configuration constant.

Your bot does NOT need to implement an authentication endpoint call. Simply attach your persistent token to every request header: Authorization: Bearer <your_access_token>.

[INTERACTIVE API DIRECTORY LINK] OPEN PARTICIPANT WEB PORTAL FOR COLORFUL, INTERACTIVE API DOCS:

URL: http://<SERVER_IP>:3003

Log into the Participant Portal in your browser and click the 'APIs' tab. You will find interactive documentation with colorful live request examples, response structures, HTTP status codes, and endpoint details. The core endpoints used by participant trading bots are listed below.

Available Participant Bot APIs (Quick Reference)

HTTP Method	Endpoint Path	Authentication	What It Does

HTTP Method	Endpoint Path	Authentication	What It Does
POST	/api/v1/orders	Bearer JWT	Submit market order: BUY, SELL, SHORT, or COVER.
GET	/api/v1/orders	Bearer JWT	Query historical order audit log, fill prices, and rejection reasons.
GET	/api/v1/portfolio	Bearer JWT	Query current cash, stock holdings, valuations, margin, and net P&L.
GET	/api/v1/portfolio/cash	Bearer JWT	Fast lightweight query to check available cash for order sizing.
GET	/api/v1/stocks	Bearer JWT	Dynamic list of all active tradable stock tickers and company descriptions.
GET	/api/v1/stocks/:ticker/history	Bearer JWT (Opt)	Historical daily OHLCV bars (anti-lookahead protected) for indicator lookbacks.
GET	/api/v1/stage	Public	Real-time platform lifecycle stage (development, preflight, simulation, status).
GET	/api/v1/simulation/clock	Public	Simulation clock: current day, total days, remaining days, stage status.
POST	/api/v1/bot/heartbeat	Bearer JWT	Send standby heartbeat sequence ACK during pre-flight readiness stage.

Historical Stock Price Data (GET /api/v1/stocks/:ticker/history)

Trading bots often require historical price lookbacks to compute technical indicators (e.g., 50-day / 200-day Moving Averages, RSI, Bollinger Bands, rolling volatility) right from Day 1 of the simulation. Instead of waiting for days of ticks to elapse, query historical daily OHLCV price bars for any tradable stock:

```
GET http://<SERVER_IP>:3002/api/v1/stocks/ALPHACORP/history?days=90
Authorization: Bearer <your_access_token>    // Optional

// Alternative query format:
// GET http://<SERVER_IP>:3002/api/v1/stocks/history?ticker=ALPHACORP&days=90

// Response:
{
  "status": "ok",
  "ticker": "ALPHACORP",
  "current_simulation_date": "2021-04-15",
  "requested_days": 90,
  "date_range": {
    "start": "2021-01-15",
    "end": "2021-04-14"
  },
  "total_records": 61,
  "prices": [
```

```
    {
      "date": "2021-01-15",
      "open": 1410.00,
      "high": 1432.50,
      "low": 1405.00,
      "close": 1422.75,
      "volume": 980000
    }
  ]
}
```

- **Query Parameters:** Specify rolling lookback in calendar days (e.g., ?days=30, ?days=90, ?days=200; default is 365). Alternatively, pass ?start_date=YYYY-MM-DD and ?end_date=YYYY-MM-DD.
- **Anti-Lookahead Guarantee:** The platform strictly enforces anti-lookahead protection. Only price bars strictly prior to the active simulation date (date < current_simulation_date) are returned. Future data is never exposed.
- **Indicator Warm-Start:** Fetch historical bars during bot startup for each discovered stock ticker to warm-start your indicator buffers.

Placing an Order (POST /api/v1/orders)

Submit orders using a JSON POST request:

```
POST http://<SERVER_IP>:3002/api/v1/orders
Authorization: Bearer <your_access_token>
Content-Type: application/json

{
  "ticker": "ALPHACORP",
  "action": "BUY",          // "BUY" | "SELL" | "SHORT" | "COVER"
  "quantity": 100,          // Positive whole integer (max 1,000,000)
  "date": "2021-04-15"      // Optional: Current simulated date
}
```

What the 4 trading actions do:

- **BUY:** Buy shares using your available cash. If you had an open short position in this stock, BUY automatically covers the short first.
- **SELL:** Sell delivery shares that you own to receive cash proceeds back into available cash.
- **SHORT:** Initiate a short sell. Borrow and sell shares at market price. Margin equal to 100% of the trade value is locked from your available cash.
- **COVER:** Cover an existing short position. Buy back borrowed shares, release locked margin, and settle profit or loss.
- **date:** Passing the current simulated date allows the system to measure your bot's decision reaction speed.

Execution Price: All orders always execute at the closing price (close price) of the current day's price candle received in the market.ticks stream.

Short Selling & Bi-Weekly Settlement Rules

- **Margin Collateral Lock:** When you open a short position, 100% of the trade value is locked from your available cash as margin collateral.

- **10-Day Settlement Square-Off:** Short positions cannot be held indefinitely. Every 10 trading days (alternate Fridays), all open short positions across all teams are automatically squared off at closing prices.
- **Profit & Loss Settlement:** Settlement profits are credited to your available cash. Settlement losses are deducted from your cash balance.
- **Automated Long Liquidation:** If a settlement loss exceeds your available cash, the platform automatically liquidates your most profitable long holdings (highest profit % first) to cover the deficit.
- **Negative Balance Block:** If your cash balance becomes negative after liquidating all stocks, your account will be blocked from placing new BUY or SHORT orders until the deficit is cleared.

Common Order Rejection Codes

Error Code	Why It Happened	What Your Bot Should Do
ACCOUNT_NOT_VERIFIED	Team has not completed verification at the registration desk.	Visit the registration desk to complete physical verification.
SIMULATION_NOT_RUNNING	Simulation is in development, paused, or not yet started.	Check clock API and wait for LIVE simulation stage.
INSUFFICIENT_CAPITAL	Order cost + brokerage fee exceeds available cash balance.	Check GET /api/v1/portfolio/cash before dispatching orders.
INSUFFICIENT_MARGIN	Short sell requires 100% margin collateral + brokerage fee.	Ensure available cash covers the full trade value.
ACCOUNT_BLOCKED	Cash balance is negative after settlement loss.	Sell existing long shares to restore a positive cash balance.
NO_SHORT_POSITION	Called COVER without holding an open short position.	Check portfolio holdings before submitting COVER orders.
INVALID_TICKER	Ticker symbol is not in active market dataset.	Use ticker symbols returned dynamically by GET /api/v1/stocks.
INVALID_QUANTITY	Quantity was zero, negative, decimal, or exceeded limit.	Submit positive whole integers only (between 1 and 1,000,000).
RATE_LIMITED	Exceeded 120 requests per minute.	Slow down order dispatch cadence; wait out 30s penalty box.

Rate Limits & Pacing

- **Request Limit:** Your team token is allowed up to 120 requests per minute.
- **Pacing:** Space out your requests. Avoid placing orders inside un-throttled tight loops.
- **Penalty Box:** Exceeding rate limits will trigger HTTP 429 and put your bot in a 30-second penalty box.

6. Development Sandbox Continuous Loop

During the Development stage (Hours 0.5 to 20), the platform replays market data in a continuous 24/7 loop so all teams can test, calibrate, and refine their bots uninterrupted.

[CONTINUOUS PRACTICE SANDBOX & RESET CASH] SANDBOX LOOP DISCONTINUITY NOTICE:

In the Development stage, when the simulation reaches the final date of the test dataset, it immediately loops back to Day 1. When this happens, market prices revert to Day 1 levels.

Any open positions bought at late-year prices may temporarily show a large unrealized P&L swing due to this price jump. This is completely expected in continuous sandbox mode.

RESET CASH DASHBOARD BUTTON: If your bot exhausts its cash or you want a fresh start during development, click the 'Reset Cash' button on your Participant Dashboard. This restores virtual testing capital back to ₹10,00,000.00.

NOTE: During the official 1-Hour Live Simulation, the dataset runs strictly ONCE without looping.

7. Simulation Parameters & Capital

- **Starting Capital:** Every team begins with exactly ₹10,00,000.00 (Ten Lakhs INR) virtual cash.
- **Time Acceleration:** 1 simulated trading day takes about 3 to 4 seconds of clock time.
- **Total Wealth Formula:** Total Wealth = Available Cash + Margin Locked + Long Market Value + Short Market Value.
- **Portfolio Valuation:** Positions are evaluated continuously at the latest closing price (close) received from Kafka.

8. Financial Mechanics & Indian Tax Regulations (IT Act)

To mirror real-world algorithmic trading and authentic market frictions, the exchange engine enforces standard brokerage, regulatory fees, and Indian capital gains tax provisions under the Income-tax Act, 1961:

- **Brokerage Fee:** Flat ₹20.00 is charged on every executed order (BUY, SELL, SHORT, COVER).
- **DP (Depository Participant) Charge:** Flat ₹15.34 is deducted when selling delivery shares.
- **Short-Term Capital Gains (STCG):** 15% on net capital gains from equity shares held 365 days or less.
- **Long-Term Capital Gains (LTCG):** 10% on net capital gains from shares held more than 365 days. Under Section 112A, the first ₹1,00,000 of LTCG per financial year is completely tax-exempt.
- **Tax Loss Offsets & Credits:** Under Sections 70 & 71, capital losses automatically offset capital gains. Long-term capital losses offset LTCG. Short-term capital losses offset STCG, and any excess STCL offsets remaining LTCG. Realized losses generate real tax credits that reduce earlier tax liabilities!
- **Annual Tax Reports:** Review your cumulative taxable gains, exemptions utilized, and net tax liability directly on your Participant Web Dashboard under the Tax & Annual Filing view.

9. Evaluation Pillars & Final Round Jury Defense

To avoid metric gaming and ensure authentic real-world engineering, the competition does NOT select winners based on a single superficial metric like gross profit. Following the 1-Hour Live Simulation, the platform evaluates all bots across automated quantitative metrics (Total Wealth, Cash Utilization, Decision Latency, Low Order Error Rates, and Tax Optimization) to qualify the finalist teams.

Finalist teams then enter the Final Round Defense (Top 5 Selection), where a distinguished jury panel conducts an in-depth code review, architectural walkthrough, and strategy defense. The jury evaluates qualifying teams holistically across five core evaluation pillars:

Core Evaluation Dimensions Considered by the Jury

- **1. Technical Implementation & System Reliability:** Working software prototype, functional correctness, code quality, proper error handling, client-side validation, clean request pacing, and resilience against network interruptions or rapid market streams.
- **2. Strategy Innovation & Analytical Depth:** Originality, creativity, and mathematical or heuristic soundness of your strategy (technical indicators, quantitative modeling, news sentiment, or statistical arbitrage). Thoughtful data utilization that extracts genuine signal rather than trading on random noise.
- **3. Execution Speed & Capital Efficiency:** Low decision latency (analyzing ticks and dispatching orders within the 3- to 4-second decision window), multi-stock concurrent throughput, efficient cash and margin deployment, and avoiding account lockouts from over-leveraging.
- **4. Market Frictions & Tax Optimization:** Prudent awareness of real-world trading costs (flat brokerage and DP charges) and deliberate planning around Indian capital gains tax provisions (STCG vs. LTCG horizons and strategic loss harvesting under Sections 70 & 71).
- **5. Team Coordination & Solution Explainability:** Clarity, structure, and confidence in demonstrating the system, balanced team contribution, and the ability to justify architectural choices, algorithmic logic, and risk controls under live questioning from the jury.

> **[JURY DEFENSE GUIDELINES]** WHAT THE JURY LOOKS FOR (BEYOND GROSS PROFIT):
>
> The jury favors disciplined engineering over unhedged gambling. A bot that produced moderate, steady gains with zero rejected orders, low decision latency, robust downside protection, and intelligent tax loss offset utilization will score significantly higher than a fragile bot that experienced massive drawdowns or relied on lucky price swings.
>
> Be prepared to open your source code, walk through your decision logic, and explain why your bot placed each trade during the live simulation run.

10. System Architecture & Getting Started Checklist

Here is the simple end-to-end architecture connecting the data stream, your bot, the gateway, and the portal:



```
| - Real-time Portfolio & P&L        |              | - Validates JWT Bearer Token       |
| - Pre-Flight Bot Readiness Status  |              | - Checks Capital & Margin Limits   |
| - Interactive API Explorer & Docs  |              | - Executes Orders at Closing Price |
| - Official Walkthrough Video Embed |              | - Deducts Brokerage, DP & Taxes    |
+------------------------------------+              +------------------------------------+
```

System Architecture Diagram

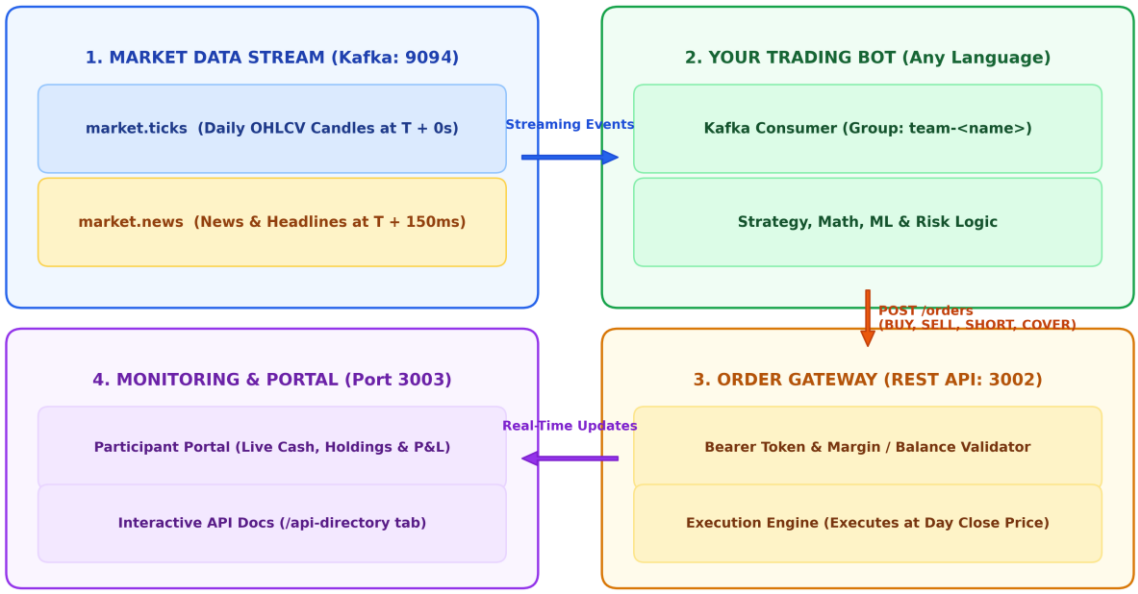


Figure 1: End-to-End System Architecture — Market Data Stream, Trading Bot, Order Gateway & Participant Portal

Appendix: 12-Step Getting Started Checklist

- **Step 1 (Mandatory):** Connect your laptop to the official venue Wi-Fi network (private competition subnet).
- **Step 2:** Open the Participant Web Portal in your browser: http://<SERVER_IP>:3003.
- **Step 3:** Register your team account directly on the Participant Web Portal.
- **Step 4:** Walk to the registration desk to complete physical team verification. Unverified teams cannot place trades.
- **Step 5:** Sign into the portal and copy your persistent Bearer access token from Step 1 of the Participant Guide UI.
- **Step 6:** Watch the official 9-minute strategy & platform briefing video embedded in the Participant Guide: https://youtu.be/QKgdReXq1-M
- **Step 7:** Review the starter code templates (Python, Node.js, Go, Java) on the portal and set up your bot environment.
- **Step 8:** Connect your bot to the Kafka broker at <SERVER_IP>:9094 using consumer group 'group-<your_username>' (scale up to 10 parallel consumers if desired).
- **Step 9:** Implement the Pre-Flight Heartbeat loop: listen for 'preflight_tick' events on 'market.ticks' and continuously ACK sequence numbers to POST /api/v1/bot/heartbeat.
- **Step 10:** Call GET /api/v1/stocks during pre-flight to dynamically load the active stock universe, and optionally GET /api/v1/stocks/:ticker/history to warm-start technical indicator buffers.

- **Step 11:** Develop and backtest your strategy in the continuous Development sandbox (use 'Reset Cash' on the dashboard if testing funds run out).
- **Step 12:** Confirm your 'Ready' status on the portal dashboard, poll GET /api/v1/stage to detect when the 1-Hour Live Simulation begins, and trade autonomously!